

Before Agents Act: A Protocol for Classifying Delegated Authority in AI Systems

Working Paper
Joshua Bloodworth
Unit27 Research
June 27, 2026

Abstract

AI agents are often governed through capability, autonomy, tool access, and runtime-control language. Those frames remain necessary and still leave a prior question unsettled. Before an agent acts, reviewers need to know what authority the system has been given inside the institution, what that authority can affect, and whether any part of it can travel through other agents or tools.

This paper introduces the Agent Authority Protocol, an open protocol proposal built around the Authority-Reach-Delegation model, or ARD. ARD classifies agent deployments with a compact $A\#-R\#-D\#$ notation across three axes: the actions or decisions an agent may perform, the people or systems it may affect, and the degree to which it may invoke, coordinate, create, or authorize other agents.

Runtime governance, IAM, policy enforcement, security review, legal review, compliance review, NIST AI RMF, OWASP guidance, ISO/IEC 42001, SOC 2 controls, and monitoring all remain necessary. ARD contributes an earlier placement record. Its purpose is to make delegated authority legible before an agent receives tools, credentials, workflow access, autonomy, or delegation rights. This paper compares ARD with adjacent governance work and tests it against cases where ordinary labels obscure the authority state reviewers need to inspect.

Keywords

AI agent governance; agentic AI; delegated authority; AI safety; agent security; delegation chains; authority classification; pre-deployment review; Unit27 Research; structural intelligence

Publication Status

This is a Unit27 Research working paper and open protocol proposal. It is not an adopted standard, certification framework, legal opinion, or compliance claim. Public references were reviewed during final preparation on June 27, 2026. The protocol should be treated as a versioned proposal open to challenge, testing, and revision.

1. Introduction

The word "agent" now covers systems that occupy very different positions inside an organization. A read-only research assistant, a content drafter, a refund agent, a coding agent with pull-request access, an autonomous remediation agent, and an agent that can create other agents may all use similar models and orchestration frameworks. They still belong to different governance categories because each one has been placed into a different operating role.

Authority makes that operating role visible. In a refund workflow, one agent may summarize the customer issue, another may recommend a refund, another may prepare the transaction for approval, another may issue refunds below a threshold, and another may authorize helper agents to process refunds during a support surge. The payment API and workflow label may look similar across those deployments. The institutional role of the agent changes in each case, and that role determines what kind of review is needed before the system should be allowed to operate.

The common language of "agent autonomy" does not carry enough structure for this problem because autonomy describes how independently the system acts without saying whether the agent is advising a human, drafting an artifact, preparing an executable action, operating inside a bounded workflow, making a material decision, or authorizing other agents to act. A system can be low-autonomy and still sit near production, finance, legal, or customer-facing consequences. A system can also be technically constrained and still create governance difficulty if it can pass work, scope, context, or authority into another actor.

Authority is the delegated permission to act inside a human and technical system. It is granted by an organization, bounded by purpose, expressed through tools and workflow placement, constrained by approvals and policies, and reconstructed through records when an action is challenged. The governance problem is structural enough that reviewers need to understand where the agent has been placed, what authority that position carries, what the agent can touch, and whether it can pass authority onward.

An agent deployment can look safe when inspected through one surface and serious when inspected through another. A service account may have access to a payment API while the agent using it is authorized only to draft a recommendation. A human may approve the final click while the agent has already selected the recipient, amount, rationale, and evidence packet. A coding agent may lack merge rights while still shaping production-relevant code. A supervisor agent may appear to be coordinating routine subtasks while creating a delegation chain that no single reviewer can reconstruct afterward.

ARD answers the placement question with the Authority-Reach-Delegation model.

A#-R#-D#

The notation is intentionally small so it can fit into READMEs, risk registers, approval records, handoff packets, incident reviews, governance documents, schemas, and review checklists. Three numbers cannot capture every risk in an agent deployment, although they can identify the authority shape early enough for reviewers to decide what additional evidence, controls, approvals, and monitoring should exist.

1.1 A Payment Agent In Four Frames

Consider a support workflow where an AI agent helps process refunds. A capability description might say the agent can read tickets, summarize customer history, interpret refund policy, draft a response, and call a payment API. A capability frame gives reviewers a performance profile while leaving unresolved whether the deployment is advising a human, preparing a transaction, executing after approval, deciding inside a threshold, or assigning work to other agents during a support surge.

An access description improves the picture by naming the systems the agent can reach, such as the CRM, policy database, support queue, and payment tool. Access still does not settle the authority grant. The same payment integration could be used by a recommendation agent that never creates a transaction, a drafting agent that prepares a refund packet for review, or a bounded operator that issues refunds below a defined limit. The tool list may look similar while the institutional permission changes.

An autonomy description adds another layer by saying whether the agent proceeds independently or waits for human approval. The approval moment can still mislead when it is treated as the whole governance question. A human may approve the final refund after the agent has already chosen the customer, amount, policy rationale, evidence summary, and customer-facing message. Even in an approval-gated workflow, the agent may shape the action path before the human sees the completed packet.

ARD classifies the same workflow by asking what authority the agent has been granted. A refund recommender may be `A1-R4-D0`. A transaction drafter may be `A2-R4-D0`. An approval-gated refund operator may be `A3-R4-D0`. A bounded autonomous refund agent may be `A4-R4-D0`. A surge supervisor that grants helper agents standing authority to process refunds may become `A6-R4-D4`.

In practical terms, a reviewer needs the authority state before choosing the review path. The same model family, tool integration, and business workflow can carry different governance obligations once the agent moves from advice to drafting, from drafting to approval-gated execution, from execution to bounded decision-making, or from decision-making to authority propagation.

Recent agent-governance work reinforces the need for this kind of prior classification. Kenney's 2026 cross-framework reference, for example, maps GDPR, the EU AI Act, NIST AI RMF, and ISO/IEC 42001 onto agent capabilities such as planning, tool use, memory, delegation, and adaptation [5]. Kenney's framework describes the compliance and runtime-control terrain around agents. ARD is complementary because it classifies the authority position of the deployment before those controls are selected.

With that placement problem established, this paper develops ARD in stages. It first separates authority from access, capability, autonomy, and runtime policy so the object of classification is clear. It then situates ARD beside adjacent work on agent governance and delegation, defines the protocol as a pre-deployment classification record across Authority, Reach, and Delegation, and tests the model against cases where familiar labels compress materially different deployments. The examples include coding agents, payment agents, and agent factories because each can appear simple under an autonomy label while occupying a different authority position.

The claim is that agent governance should classify authority before it classifies autonomy. If authority remains implicit, teams may substitute tool lists, access scopes, autonomy levels, safety policies, or audit logs for the central question. Those artifacts remain necessary. By themselves, they do not say what

institutional permission has been handed to the agent. ARD gives that question a working form before the system becomes operational.

2. Authority as a Governance Object

Authority is the organizational permission to act, affect, decide, or pass permission onward. It is not identical to model capability, technical access, runtime autonomy, or policy enforcement, although each one exposes part of the deployment surface. ARD makes the authority assignment explicit enough that the surrounding governance stack can be configured against it.

Framed this way, authority is a reviewable claim about placement. It tells a reviewer which institutional role the agent occupies before the reviewer studies model performance, prompt design, or specific tool scopes.

Access describes what a system can reach, while authority describes what the system is allowed to do with that reach. A service account may be able to call a payment API while the agent using it is authorized only to prepare a draft recommendation. The technical access is real, but the authority grant is narrower. If the review records only the service account and tool list, it misses the operating promise that the deployment is making.

Capability has the same problem from the other direction. A model may be capable of writing persuasive customer emails, generating code, selecting vendors, drafting legal language, or planning a remediation sequence. Authority describes which of those outputs may enter a live workflow, which human or policy gate controls that entry, and what happens if the output becomes an action. A capability taxonomy can tell reviewers what the agent might do. An authority taxonomy tells reviewers what the deployment has been allowed to do.

Autonomy is also insufficient as the main governance object. An approval-gated agent can prepare an action so completely that the human reviewer is left confirming a path the system already assembled. A bounded autonomous agent may act without human approval while remaining inside a narrow, reversible workflow. A delegating agent may not touch the external system directly, yet it may alter the actor set by assigning work to other agents. In each case, the governance significance depends on the authority position as well as the degree of independence.

Runtime policy describes what will be blocked, allowed, routed, monitored, or logged. Authority describes the permission structure those runtime choices are meant to enforce. During review and incident analysis, the separation has operational consequences. Logs can show what happened, and policy engines can show which rule fired, but neither automatically explains whether the agent should have occupied that role in the first place.

The same technical surface can carry different governance meanings. A coding agent that drafts a patch, opens a pull request, merges to production, rolls back a failed deployment, or creates other agents with repository access is moving through different authority states. Calling each one a "coding agent" hides the boundary that reviewers need to see, since autonomy is only one part of the review. The review question is what role the agent has been given in the action path.

The role can remain invisible when organizations describe agents through tools or workflows alone. A prompt, model name, service account, orchestration graph, or approval gate may tell part of the story,

but none of those artifacts automatically records the socially and operationally meaningful permission the agent holds. Authority is the missing object that connects those artifacts into a governance claim.

Once authority is named, the review target changes. The reviewer can move beyond what the model can do or what the tool can reach and ask what the agent has been placed in position to do, who approved that placement, what evidence will exist if the action is challenged, and whether later agents or tools can inherit any part of that authority. The review now needs institutional design language in addition to tool-permission language.

3. Why Classification Belongs Before Deployment

Authority is easiest to inspect before deployment, while the proposed system is still close to its design documents, access decisions, approval path, and operating assumptions. At that stage, reviewers can usually identify the agent's purpose, action rights, affected systems, approval gates, helper agents, delegation paths, evidence requirements, and revocation paths. After deployment, the same facts may be scattered across prompts, tool manifests, logs, access-control records, workflow settings, incident traces, and informal team memory.

Agent risk often turns on placement as much as model behavior, which makes classification strongest before the placement hardens into production practice. A customer-email drafter that waits for human review, a pull-request agent without merge rights, a refund recommender, and a temporary helper inside a bounded workflow occupy different positions from systems that can send customer emails directly, merge to production, issue refunds, or grant standing authority to other agents. These deployments create different pre-deployment obligations. Reviewers have to know whether the agent is assisting, preparing, acting, deciding, or authorizing before they choose evidence, approval, monitoring, or revocation controls.

These boundaries are easy to blur once every system is called an agent. The blur can then survive into control design. A team may build strong monitoring around tool calls while leaving the agent's role unclear. Another team may rely on human approval without asking what the agent has already selected, filtered, summarized, or prepared before the approval moment. A third team may allow subagents as an orchestration convenience without recording whether authority, scope, or accountability traveled with the task.

ARD names those boundaries before they become part of the operating environment. The classification does not certify the system as safe. It creates a shared starting point for deeper review by naming the authority shape reviewers are being asked to accept.

The governance sequence changes. Reviewers classify the agent's authority, reach, and delegation, then decide what policy, monitoring, approval, testing, logging, and incident review are appropriate for that classification. In this sequence, ARD supplies the authority record runtime enforcement and compliance mapping need in order to know what they are enforcing.

For that reason, ARD treats classification as a record, not a label. A label can be copied into documentation and forgotten. A record can be challenged when tools change, when reach expands, when a human approval gate is removed, when a helper agent is added, or when a deployment begins

to pass work to other agents. The pre-deployment record gives later reviewers a baseline against which drift can be seen.

4. Related Work

ARD sits beside existing governance, security, and authorization work. Its contribution is narrower than those adjacent systems because it focuses on the moment before runtime controls, compliance mapping, or incident review begin, when reviewers still need a compact way to name delegated authority.

Microsoft's Agent Governance Toolkit addresses a broad runtime governance surface, including policy enforcement, zero-trust identity, execution sandboxing, and reliability engineering for autonomous agents [1]. NIST AI RMF 1.0 provides a voluntary framework for managing AI risks across design, development, deployment, and use [2]. ISO/IEC 42001 defines an AI management system standard for establishing, implementing, maintaining, and improving AI management systems [4]. OWASP's agentic security work focuses on risks in autonomous agents and multi-step AI workflows [3]. Feng et al. formalize levels of autonomy for AI agents [6]. South et al. focus on authenticated delegation and authorized AI agents [7]. Ibrahim and Li treat delegation and scope as a compositional authorization problem in agentic AI [8]. Patil frames delegation chains as a security object for federal multi-agent systems [9].

ARD also sits beside long-running access-control and IAM practice, including role-based access control, attribute-based access control, capability-based security, least privilege, service accounts, and OAuth or OpenID Connect scopes. Those systems define or enforce who or what may use a resource under particular conditions. ARD addresses an earlier review artifact: the intended authority shape of an agent deployment before credentials, scopes, policies, or delegation chains are configured. ARD is a placement record rather than an enforcement substitute. It can feed enforcement systems, audit envelopes, and compliance mappings.

Kenney's 2026 cross-framework reference sits close to this problem space because it treats AI agents as a distinct governance category once systems can plan, call tools, retain state, and delegate work [5]. Kenney maps agent capabilities to GDPR, the EU AI Act, NIST AI RMF, ISO/IEC 42001, and runtime enforcement patterns such as tool gating, memory scoping, delegation controls, and audit envelopes. The overlap is real, while the center of gravity is different. Kenney answers how agent capabilities should be governed through compliance analysis and runtime controls. ARD answers which authority grant those controls are supposed to protect.

Feng et al. are the nearest neighbor for ARD's Authority axis because their autonomy levels describe how human responsibility shifts as agent independence increases [6]. ARD treats independence as one part of the authority state when independence changes what the deployment may carry into an action path and what evidence reviewers need. The Authority axis therefore names a granted action role rather than a general measure of model independence.

The authorization and delegation papers are close neighbors for the Delegation axis. South et al. and Ibrahim and Li work near the technical layer where authenticated agents, scopes, and delegation rules are expressed [7,8]. Patil turns delegation chains into a security object for high-accountability multi-

agent systems [9]. ARD sits in front of that machinery by classifying the authority state that those mechanisms should preserve, constrain, or reject.

Tool gating, audit envelopes, decision records, and policy-as-code controls all need a target state. They can record or enforce whether a tool call was permitted, whether memory was scoped, whether a delegation chain existed, and whether evidence can be reconstructed. They do not automatically say whether the deployment should have held that authority before the action occurred. ARD supplies that prior authority record.

ARD asks an earlier placement question about what authority this agent is supposed to hold in the first place. The answer changes how the rest of the stack should be configured. A read-only analyst, an approval-gated production operator, and a delegating agent may all use the same model family and orchestration layer. Each should produce a different approval record, monitoring plan, revocation path, or incident review standard.

Adjacent work	Primary focus	ARD relationship
NIST AI RMF	Broad AI risk management	ARD can supply an authority record for agent-specific risk review.
ISO/IEC 42001	AI management systems	ARD can support documentation of agent scope, accountability, and review.
OWASP Agentic Security Initiative	Agentic security risks and threat categories	ARD can help scope which risks should be prioritized for a given authority profile.
Microsoft Agent Governance Toolkit	Runtime governance, identity, policy, sandboxing, reliability	ARD can classify authority before runtime governance controls are selected.
Kenney's agent governance crosswalk	Cross-framework compliance mapping, runtime enforcement, tool gating, memory scoping, delegation controls, audit envelopes	ARD can supply a prior authority record for the controls and evidence envelopes that govern live agent behavior.
Levels of autonomy research	Human responsibility as agent independence increases	ARD treats autonomy as one input while classifying the agent's granted action role.
Access control and IAM	Permissions, identity, scopes, credentials, least privilege	ARD records intended authority before credentials, scopes, or policies are granted.
Authenticated delegation and delegation-chain research	Identity, authorization, propagation, scope, auditability	ARD can define the authority that technical delegation systems need to represent.

ARD is upstream of several adjacent control surfaces. It supplies a small authority record that later systems can use while they determine the rest of the risk posture. The notation also needs to be read carefully against adjacent taxonomies. In this paper, **A** means Authority. It does not mean capability,

agency, or autonomy. Other agent-governance work may use [A1](#), [A2](#), and similar labels for capability classes; ARD uses the [A](#) axis only to classify the authority an agent has been granted.

5. What ARD Catches

ARD should change the review when ordinary descriptions sound adequate. Capability, autonomy, access, and runtime-control language each reveal part of an agent deployment. The failure appears when one of those descriptions is asked to carry the whole governance burden.

A capability description tells reviewers that an agent can plan, use tools, retain state, or delegate. It may still leave open whether the agent is allowed to act on those capabilities. An autonomy description tells reviewers how independently the system proceeds. It may still hide whether the system is near money, production, customers, legal records, or downstream delegation. A tool-access description tells reviewers what the system can reach. It may still miss the action rights attached to that reach. Runtime controls tell reviewers what the system will allow, block, route, monitor, or record. They may still assume the authority state that should have been classified before enforcement.

ARD earns its place when it changes what a reviewer asks for next.

Case	Ordinary description	What ARD changes
Approval-gated payment action	Human-in-the-loop, low-autonomy workflow	A3-R4-D0 keeps financial reach visible. Review shifts toward transaction payload, approval record, action receipt, and reversal path.
Local autonomous formatting agent	Autonomous agent running without step-by-step approval	A4-R0-D0 keeps autonomy from inflating the review. Review can focus on local sandboxing, version history, and recovery rather than financial or production controls.
Same payment API, different role	Agent has access to the payment tool	A2-R4-D0 , A3-R4-D0 , and A4-R4-D0 are different authority states. Drafting, approval-gated execution, and bounded autonomous execution need different evidence.
Supervisor agent with helpers	Multi-agent orchestration	D1 , D2 , D3 , and D4 separate approved invocation, orchestration, temporary agent creation, and authority propagation. Review shifts toward delegation graph, inherited scope, expiration, and revocation.

These comparisons do not prove that ARD is complete. They show the narrower claim: an authority classification can make two similar-looking deployments require different evidence, and two dramatic-sounding deployments require different levels of concern. For a working protocol, the narrower claim is enough.

6. The ARD Model

ARD classifies an agent along three axes that together describe the authority shape of a deployment.

[A#-R#-D#](#)

Where:

- A means Authority
- R means Reach
- D means Delegation

An example classification is:

A3-R4-D1

This means:

- A3: the agent may prepare and execute actions only after explicit human approval
- R4: the agent can affect financial systems or decisions
- D1: the agent may invoke approved helper agents or tools inside a fixed workflow

The axes are separated because they answer different parts of the placement question. Authority names the kind of action the agent may perform. Reach names the people, systems, records, or domains that action may affect. Delegation names whether the agent can involve other agents in a way that expands the review surface. A low-autonomy agent may still have high reach, and a narrow local agent may become higher risk if it can create or authorize other agents.

6.1 Axis Independence

The three axes are independent and related. The Authority axis records the kind of authority the agent itself may exercise. The Reach axis records the domain affected by that authority. The Delegation axis records whether work, scope, context, or authority can move across actor boundaries.

This separation prevents ordinary helper use from being mistaken for delegated authority. An A4-R3-D1 customer-support agent may act directly in a customer-facing workflow and call an approved helper inside that workflow. It is not an A6 agent if it cannot decide who else may act, create agents with new scopes, modify another actor's authority, or authorize another agent to carry authority forward.

The same separation protects the upper end of the ladder. D1 and D2 can exist without A6 because approved invocation and orchestration may be fixed by design. At the upper end, A6 and D4 are related without being identical. A6 describes discretionary authority over the actor set: the agent can decide who or what may act next, alter scope, grant credentials, or change another actor's authority. D4 describes a delegation path in which authority can propagate, whether that propagation is discretionary, policy-bound, or approval-gated. An approval-gated system might prepare a delegation packet for human approval, which raises Delegation to D4 without giving the agent full A6 discretion. A system that can choose the new actor's scope, credentials, persistence, or action rights has entered A6 territory.

The review question is whether the first agent can change who or what is authorized to act next.

7. Authority Classes

Authority describes what decisions or actions an agent may perform. The ladder moves from observation to advice, drafting, approval-gated execution, bounded autonomous operation, operational decision-making, and authority propagation.

The Authority cuts follow the action path. Each rung adds a different form of participation: observe, recommend, create an artifact, prepare an executable action for approval, execute inside limits, make bounded operational choices, or alter the authority of other actors.

Class	Name	Description
A0	Observer	Read-only. Cannot recommend, draft, modify, or act.
A1	Advisor	Can analyze and recommend. Human decides and executes.
A2	Drafter	Can create artifacts for human review. Cannot publish, send, merge, or execute.
A3	Approval-Gated Operator	Can prepare and execute actions only after explicit human approval.
A4	Bounded Autonomous Operator	Can execute predefined workflows independently within strict limits.
A5	Autonomous Decision Maker	Can make bounded operational decisions with material external or business impact.
A6	Delegating Authority	Can decide who or what may act next by determining, granting, changing, or propagating authority for other agents or actors.

The lower classes separate forms of human dependence that are often collapsed together. [A0](#) and [A1](#) allow the agent to observe, analyze, or recommend without creating an artifact that can be mistaken for operational action. [A2](#) introduces artifact creation, which means reviewers need provenance: what was drafted, what was changed, who accepted it, and where it entered a live workflow.

[A3](#) changes the action path because the agent can prepare an executable action and carry it to the point of human approval. The approval gate remains central, but the agent is now part of the operational chain. [A4](#) removes repeated human approval inside a bounded workflow, so limits, thresholds, exception handling, and rollback paths carry more review weight than the simple presence or absence of a person in the loop.

[A3](#) also requires special evidence because approval can hide pre-assembly. Reviewers need to know what the human actually saw. That includes the generated action, preselected fields, evidence packet, alternatives presented or omitted, and what remained editable before approval.

[A5](#) gives the agent bounded decision authority with material external or business impact. At this level, the agent is making operational choices that may affect customers, money, production systems, legal records, or business continuity. [A6](#) is qualitatively different because it concerns authority over other agents or actors. A system at this level can change who or what is authorized to act next, so the review surface includes delegation paths, inherited scopes, expiration rules, revocation paths, and records of who or what authorized the next actor.

8. Reach Classes

Reach describes what people, systems, data, or business domains an agent can affect. It is framed as a pre-deployment property because a team can usually know what category of thing the agent can touch before it knows the eventual impact of any particular action.

The Reach cuts follow the affected domain rather than the eventual harm. Local files, internal content, internal systems, external people, money, legal or compliance records, and production or critical operations create different review obligations even before a particular incident has occurred.

Class	Name	Description
R0	Local / No External Effect	No external systems, users, customers, or persistent records affected.
R1	Internal Content	Can affect internal documents, drafts, notes, or knowledge artifacts.
R2	Internal Systems	Can affect internal workflows, tickets, tasks, or operational systems.
R3	Customer-Facing	Can affect customers, users, external communication, or public content.
R4	Financial	Can affect invoices, payments, refunds, pricing, procurement, or budgets.
R5	Legal / Compliance	Can affect contracts, regulated records, compliance decisions, or formal attestations.
R6	Production / Critical Operations	Can affect production systems, deployments, infrastructure, safety-critical operations, or high-impact business continuity.

Reach should be classified by the highest-risk domain the agent can affect through its tools, credentials, outputs, or workflow placement. This axis prevents reviewers from treating autonomy as the only source of seriousness. A human-approved coding agent with production deployment reach deserves a different review from an autonomous local formatting agent because the deciding factor is what the agent's action can touch as well as how independently it acts.

Reach also avoids waiting for impact language to arrive too late. A customer-facing email may have no consequence, or it may trigger reputational, contractual, or support risk. A small code change may be harmless, or it may affect production reliability. A pricing change may be routine, or it may create downstream financial exposure. Because impact depends on context and outcome, Reach begins with the more inspectable question of what category of thing the agent can affect.

Internal systems deserve the same discipline because tickets, task queues, knowledge bases, CRM records, and operational dashboards can become decision surfaces. If an agent can alter those surfaces, it may affect the humans and systems downstream from them even when it never contacts a customer directly.

9. Delegation Classes

Delegation describes whether an agent can invoke, coordinate, create, or authorize other agents. It is the axis most likely to be under-specified in early deployments because subagents can look like implementation details until they begin to carry work, context, scope, or authority across actor boundaries.

The Delegation cuts follow how the actor set changes. Fixed helper use, orchestration among approved actors, temporary actor creation, and authority propagation create different reconstruction burdens after an action is challenged.

Class	Name	Description
D0	No Delegation	Cannot call, invoke, create, or direct other agents.
D1	Approved Invocation	May invoke pre-approved agents or tools within a fixed workflow.
D2	Agent Orchestration	May coordinate multiple approved agents within a defined workflow.
D3	Temporary Agent Creation	May create temporary agents for bounded tasks, without granting persistent authority.
D4	Authority Delegation	May assign authority, pass credentials, modify scopes, or authorize other agents.

The delegation ladder starts with ordinary non-delegation, then moves into approved helper invocation, orchestration, temporary agent creation, and authority delegation. D1 remains narrow because the helper set and workflow are known. D2 introduces compound behavior, sequencing, and responsibility across multiple actors. D3 creates a new actor for a bounded task, which requires reviewers to know what the actor could access, how long it existed, what it produced, and how it was retired.

D4 is the highest-risk class because it allows authority to propagate. At that level, the relevant object is an authority chain. Reviewers need chain-specific evidence: delegation graph, inherited scope, expiration, revocation, approval chain, and a record of which actor authorized which later actor. Without that evidence, it becomes difficult to determine whether an eventual action was authorized or merely reachable.

10. Risk Banding Heuristic

ARD produces a first-pass risk band from the classification. The risk band is a conservative review heuristic for determining evidence, approval, monitoring, and revocation requirements. It is not a statistical risk estimate and should not be read as proof of safety.

Each row below is an escalation trigger. Use the highest row triggered by Authority, Reach, or Delegation. Reach and Delegation can raise the band, and neither should lower the band implied by Authority.

Escalation trigger	Risk Band
All of A0-A1, R0-R1, and D0	Minimal
Any A2, R2, or D1, absent a higher trigger	Low
Any A3, R3, or D2, absent a higher trigger	Moderate
Any A4, R4, or D3, absent a higher trigger	High
Any A5, R5, R6, or D4, absent a higher trigger	Critical
Any A6, or R6 with D4	Extreme

The heuristic uses three simple rules. First, start with the authority class because it describes the action rights the system has been granted. Second, raise the review band when Reach moves the action into customer-facing, financial, legal, compliance, production, or critical operations domains. Third, raise the review band when Delegation makes responsibility harder to reconstruct or allows authority to propagate.

Rule	Review effect
Higher Authority raises baseline seriousness	Drafting, approval-gated execution, autonomous operation, and decision-making produce different evidence expectations.
Higher Reach raises consequence exposure	Customer-facing, financial, legal, compliance, and production domains require stronger approval, logging, rollback, and incident paths.
Higher Delegation raises reconstruction burden	Helper invocation, orchestration, temporary agent creation, and authority propagation require stronger chain evidence.
Highest-risk dimension dominates	A low-autonomy agent with financial reach or delegation authority may require a higher review band than autonomy alone suggests.

The highest-risk dimension should dominate review because an agent with modest autonomy may still require high-risk controls when it has legal, financial, production, or delegation reach. A local agent may also move into a higher band if it can create agents or coordinate multi-agent behavior in ways that make responsibility harder to reconstruct. The table should therefore be read as a review escalation guide, not as a closed mathematical scoring system.

11. Evidence Requirements

ARD ties classification to evidence expectations because higher authority, reach, or delegation classes require reviewers to reconstruct more of the authority state that existed at the time of action.

Classification Pattern	Evidence Required
A0-A1 / R0-R1 / D0	Basic logs and source references
A2 / R1-R2 / D0-D1	Draft artifact, human review record, version history
A3 / R2-R4 / D0-D1	Tool-call trace, approval packet shown to the human, preselected fields or alternatives, approval record, action receipt
A4 / R2-R4 / D1-D2	Runtime logs, policy decision log, exception log, rollback path
A5 / R4-R6 / D1-D3	Full audit trail, human checkpoint policy, incident path, periodic review
A6 or D4	Delegation graph, inherited scopes, approval chain, revocation path, authority expiration

Evidence is where the classification becomes operational, since an agent that sends a message, issues a refund, opens a pull request, changes infrastructure, or delegates authority to another agent should leave enough record for a reviewer to answer what happened, what authority was in force, who approved the authority, and what path existed for rollback or revocation.

12. Worked Examples

The following examples show how ARD separates agents that may look similar under capability or autonomy labels.

Agent	Classification	Governance signal
Read-only research agent	A1-R0-D0	It can advise without affecting external systems or passing work onward.
LinkedIn content drafter	A2-R1-D0	It can create public-facing raw material while human review controls publication.
Customer support responder	A4-R3-D1	It may act directly in a customer-facing channel and call approved helpers.

Agent	Classification	Governance signal
GitHub coding agent with PR approval	A3-R6-D1	It is approval-gated and its reach includes production-critical code paths.
Finance agent preparing payment recommendations	A3-R4-D0	Human approval remains, and financial reach raises the review band.
Autonomous DevOps remediation agent	A5-R6-D2	It can make operational decisions in production and coordinate approved agents.
Agent factory with production authority	A6-R6-D4	It can propagate authority in a critical operational domain.

The examples follow the same pattern: identify the agent's position in the action path, identify what the action can affect, and then ask whether authority can travel through other agents or tools. The classification does not end the review, although it gives reviewers a better starting point.

12.1 Coding Agent With Pull-Request Approval

A coding agent that opens pull requests without merge rights may appear modest because a human still reviews and accepts the final change. ARD would still classify it with high reach if the repository affects production systems. The authority class may be A3, because the agent prepares and executes an action after approval. The reach may be R6, because the code path can affect production or critical operations. Delegation may be D1 if the agent can call approved helper tools or test agents inside a fixed workflow.

The resulting classification, A3-R6-D1, keeps "approval-gated" separate from "low risk." The human approval gate remains central, yet the agent is still operating in a production-relevant action path. Evidence should include the generated diff, test output, tool-call trace, approval record, merge actor, and rollback path.

12.2 Refund Or Payment Agent

A finance-adjacent agent shows why reach deserves its own axis. An agent that prepares refund recommendations may be A1 or A2 depending on whether it only advises or drafts a transaction artifact. An agent that prepares and submits a refund after explicit approval may be A3-R4-D0. An agent that issues refunds below a threshold without review may be A4-R4-D0.

If the threshold rule is mechanical and the agent only checks predefined conditions, A4 is appropriate. If the agent chooses whether the refund should be issued under ambiguous policy, weighs customer value against financial exposure, or resolves exceptions without approval, the class rises to A5.

Those systems may use the same customer data and payment tooling, while their authority profiles differ. The relevant question is whether the agent can move money, prepare a money-moving action, or cause a human to approve one with incomplete context. Evidence should change with the classification: source references and review history for a recommendation, an approval record and action receipt for

an approval-gated payment action, and runtime logs, threshold policy, exception logs, and remediation paths for bounded autonomous payment action.

12.3 Agent Factory

An agent factory is the clearest case for separating delegation from autonomy. A system that creates agents for bounded local tasks may be high risk and still containable if the created agents have no persistent authority and expire cleanly. A system that creates agents and grants standing authority has a different authority profile.

An **A6-R6-D4** agent factory with production authority is a structure for generating future actors with operational power. The review target is the factory, the agents it creates, the authority they inherit, the scope they receive, the expiration of that scope, and the path for revoking it.

In this case, the dossier should describe the full delegation chain. Reviewers should be able to answer which agent created which actor, under what authority, with what scope, for how long, and with what revocation path.

13. Protocol Artifacts

The public Agent Authority Protocol repository includes:

- class definitions for Authority, Reach, and Delegation
- a risk banding guide
- an Agent Authority Dossier template
- evidence requirements
- a YAML risk scorecard
- a delegation-chain schema
- worked examples
- related-work mappings
- public claim boundaries
- a release verifier

The Agent Authority Dossier is the primary review artifact. It records the agent identity, purpose, ARD classification, tools and systems, data access, action rights, approval gates, evidence and logging, revocation and recovery path, and residual risk.

The dossier is intentionally mundane, and that plainness is part of the design. A governance artifact should survive contact with real deployment work, including pull requests, approvals, security reviews, compliance notes, incident review, and handoff between teams.

14. Discussion

ARD makes the operating shape of authority legible before authority becomes action. For agents, that structure includes who or what can act, what the action can affect, how approval is represented, where evidence will live, and whether authority can pass into another actor.

The model is deliberately plain because many agent failures will come from ordinary organizational ambiguity: unclear approval rights, poorly scoped tools, inherited credentials, untracked delegation, and systems that treat "the agent did it" as an explanation. ARD gives those problems a name early enough to change deployment decisions.

ARD's claim remains modest because it is a classification proposal, not a full governance regime. The case for ARD comes from the way agents change the cost of organizational action. When drafting, routing, deciding, escalating, and coordinating become cheaper, boundary-setting becomes more consequential. A team may be comfortable letting an agent draft one email, open one ticket, or propose one patch. The governance problem changes when those actions become continuous, customer-facing, financial, production-relevant, or delegable.

The protocol is intentionally legible to non-specialists, so a security team, product team, compliance reviewer, engineering manager, operator, or executive should be able to read [A3-R6-D1](#) and ask better questions about the approval gate, the reason for production reach, the helper agents the system can invoke, and the logs that prove the action stayed inside scope. A small classification language changes the first review conversation before deeper systems do their work.

The design commitments are:

1. Classify before deployment.
2. Treat autonomy as one input to authority classification.
3. Separate reach from impact.
4. Treat delegation as a distinct axis.
5. Let the highest-risk dimension dominate review.
6. Tie classification to evidence requirements.
7. Preserve revocation and expiration paths for high-risk authority.

This design applies most directly to multi-agent systems, coding agents, customer-facing agents, finance agents, compliance agents, production remediation agents, and agent factories.

15. Limitations

The Agent Authority Protocol has several limits. First, it is a proposal and has not been adopted by a standards body. Second, ARD classification depends on accurate knowledge of the agent's tools, credentials, workflows, and deployment environment. A classification can be wrong when the underlying system description is wrong. Third, the model is coarse by design. Domain-specific risk analysis, threat modeling, legal review, compliance mapping, runtime policy enforcement, monitoring, and incident response planning remain necessary.

The risk bands are heuristic. They help scope review without functioning as statistical risk estimates or safety certifications. ARD should be treated as a classification layer that feeds technical controls, approval gates, logging, revocation, and monitoring. The model may also need domain-specific profiles before it can be applied cleanly in regulated settings, safety-critical settings, or high-complexity multi-agent systems.

ARD has not yet been tested for inter-rater reliability. The worked examples are illustrative, and reasonable reviewers may disagree at class boundaries until domain profiles, calibration examples, and decision rules exist. That makes early use of ARD strongest as a structured review prompt rather than a conformance claim.

ARD also depends on institutional honesty. The predictable failure mode is under-classification: a team may call an **A5** system **A3**, omit **R4** or **R6** reach, or describe **D4** authority propagation as helper invocation. A clean dossier can hide a bad deployment when the record understates tool access, reach, approval gates, delegation paths, or the practical effect of generated artifacts. ARD should therefore be paired with challenge rights. Reviewers need permission to contest a classification, request evidence, require high-reach or high-delegation sign-off, and revise the record when the deployed system drifts.

16. Future Work

Future work should move in three directions.

First, ARD needs stricter machine-readable forms. A formal JSON Schema, conformance profiles for common agent categories, versioned classification records, delegation graph validation, and a small CLI would make dossiers easier to apply and inspect across real deployments.

Second, ARD should be mapped into live governance systems. Useful next steps include integration examples for runtime governance tools, mappings from ARD dossiers to audit envelopes, decision records, and policy-as-code controls, and mappings to OWASP Agentic Application risks.

Third, the protocol needs adversarial use. Case studies should compare ARD classifications across similar agents with different authority profiles, and review exercises should test inter-rater reliability and whether teams understate authority, reach, or delegation. Regulated domains, safety-critical settings, production remediation agents, and agent factories would be strong test cases.

17. Conclusion

AI agent governance needs a clearer language for delegated authority. Before agents receive tools, credentials, workflow access, autonomy, or delegation rights, reviewers need a compact way to describe what authority has been handed to the system, what the system can affect, and whether authority can propagate further.

The Agent Authority Protocol proposes ARD as that pre-deployment classification layer. Its intentionally small structure includes Authority, Reach, Delegation, risk banding, evidence requirements, and a dossier template. The test is whether reviewers can identify what authority an agent has been given before asking whether it should be monitored, constrained, certified, insured, audited, or trusted.

If agents are going to act inside institutions, authority cannot remain implicit. It should be treated as a structural assignment made before action begins, then carried into the controls, records, and reviews that follow. ARD's purpose is to make authority legible before it becomes action.

References

- [1] Microsoft. "Agent Governance Toolkit." GitHub. <https://github.com/microsoft/agent-governance-toolkit>
- [2] National Institute of Standards and Technology. "Artificial Intelligence Risk Management Framework (AI RMF 1.0)." <https://www.nist.gov/itl/ai-risk-management-framework>
- [3] OWASP Gen AI Security Project. "Agentic Security Initiative." <https://genai.owasp.org/initiatives/agentic-security-initiative/>
- [4] International Organization for Standardization. "ISO/IEC 42001:2023, Artificial intelligence - Management system." <https://www.iso.org/standard/42001>
- [5] Kenney, N. M. "Governing Agents: A Practitioner's Cross-Framework Reference. Mapping GDPR, the EU AI Act, NIST AI RMF, and ISO/IEC 42001 to AI Agents." First Edition, Digital 520, 2026.
- [6] Feng, K. J. K., McDonald, D. W., and Zhang, A. X. "Levels of Autonomy for AI Agents." arXiv:2506.12469, 2025. <https://arxiv.org/abs/2506.12469>
- [7] South, T., Marro, S., Hardjono, T., Mahari, R., Deslandes Whitney, C., Greenwood, D., Chan, A., and Pentland, A. "Authenticated Delegation and Authorized AI Agents." arXiv:2501.09674, 2025. <https://arxiv.org/abs/2501.09674>
- [8] Ibrahim, A., and Li, Y. "Overlaying Governance: A Compositional Authorization Framework for Delegation and Scope in Agentic AI." arXiv:2606.03518, 2026. <https://arxiv.org/abs/2606.03518>
- [9] Patil, K. "SentinelAgent: Intent-Verified Delegation Chains for Securing Federal Multi-Agent AI Systems." arXiv:2604.02767, 2026. <https://arxiv.org/abs/2604.02767>
- [10] Unit27 Research. "Agent Authority Protocol." GitHub. <https://github.com/unit27research/agent-authority-protocol>